

Guia de Estudo e Revisão: Algoritmos e Programação com Python (Aulas 03-13)

Este documento consolida os conceitos fundamentais, estruturas de dados e metodologias de resolução de problemas abordados ao longo do curso. Ele serve como um roteiro lógico para transformar lógica pura em código Python robusto e eficiente.

1. Fundamentos da Resolução de Problemas

Programar não é apenas digitar código, mas traduzir a lógica humana para uma linguagem que a máquina compreenda. Para evitar confusões, adotamos uma abordagem sistemática.

Modelo IPO (Entrada, Processamento, Saída)

Antes de iniciar qualquer algoritmo, decompõe o problema nestas três camadas essenciais:

Componente	Descrição
Entrada (Input)	Identifica quais dados o programa recebe, sua origem e a ordem de entrada.
Processamento	Define as transformações, cálculos, filtros e regras de negócio aplicadas aos dados.
Saída (Output)	Determina qual o resultado final esperado e como ele deve ser apresentado ao usuário.

Roteiro de 5 Passos para Resolução

- Identificar o Modelo IPO:** Descreva o fluxo de dados sem se preocupar com a sintaxe técnica.
- Escolher o Laço de Repetição:** Defina se a repetição é sobre uma coleção conhecida (for) ou baseada em uma condição que precisa ser atingida (while).
- Mapear o Ciclo de Vida das Variáveis:** Determine quais variáveis persistem durante todo o programa e quais precisam de "reset" a cada iteração.
- Decompôr em Funções:** Uma técnica valiosa é sublinhar os verbos do enunciado; **cada verbo diferente é um forte candidato a tornar-se uma função** com responsabilidade única.
- Escrever o Programa Principal e Testar:** Monte a estrutura final e valide com casos de borda (entradas mínimas, limites exatos e listas vazias). **Regra de Ouro:** Se você não consegue explicar a solução em português, você ainda não está pronto para codificar.

2. Tipos de Dados, Variáveis e Operadores

Tipos Primitivos

Tipo, Definição, Exemplo

int, Números inteiros (positivos ou negativos)., "42, -7"

float, Números com parte decimal (ponto flutuante)., "3.14, 10.0"

str, Cadeias de caracteres (textos) entre aspas., "''Python''", "'Aula 03'"

bool, Valores lógicos (Verdadeiro ou Falso)., "True, False"

Variáveis e Boas Práticas

- snake_case:** Nomes de variáveis devem ser escritos em minúsculas, com palavras separadas por sublinhado (ex: `media_final`, `total_vendas`).

- **Nomes Descritivos:** Evite nomes como x ou y. Use nomes que revelem o conteúdo da variável.
- **Atribuição:** O símbolo = não significa igualdade matemática, mas sim que a variável à esquerda está recebendo o valor à direita.

Operadores

Aritméticos | Operador | Descrição | Exemplo de Código | | :--- | :--- | :--- | | + / - | Soma / Subtração | 5 + 2 resulta 7 | | * / / | Multiplicação / Divisão | 10 / 4 resulta 2.5 | | // | Divisão Inteira | 10 // 4 resulta 2 | | % | Resto da Divisão | 10 % 3 resulta 1 | | ** | Potência | 2 ** 3 resulta 8 | **Dica do Professor:** Em Python, o operador de divisão / **sempre** retorna um valor do tipo float, mesmo que a divisão seja exata.

Relacionais (Comparam valores) | Operador | Descrição | Exemplo de Código | | :--- | :--- | :--- | | == / != | Igual / Diferente | 5 == 5 (True) | | > / < | Maior / Menor | 10 < 3 (False) | | >= / <= | Maior ou igual / Menor ou igual | 18 >= 18 (True) |

Lógicos (Combinam condições) | Operador | Descrição | Exemplo de Código | | :--- | :--- | :--- | | and | True apenas se ambas forem verdadeiras. | (5 > 3) and (2 < 4) (True) | | or | True se pelo menos uma for verdadeira. | (5 < 3) or (2 < 4) (True) | | not | Inverte o valor lógico (negação). | not (x > 10) |

3. Estruturas de Decisão (IF, ELSE, Match)

O fluxo do programa é controlado por condições que determinam qual bloco de código será executado.

- **if:** Executa o bloco se a condição for verdadeira.
- **if/else:** Oferece um caminho alternativo para quando a condição é falsa.
- **if/elif/else:** Utilizado para múltiplas faixas de valores ou condições sequenciais.

```
# Exemplo de Classificação de Temperatura
temp = float(input("Temperatura: "))
if temp > 30:
    print("Quente")
elif temp >= 20:
    print("Agradável")
else:
    print("Frio")
```

Seleção por Padrão (Match/Case)

Introduzido no Python 3.10, o match/case é a escolha ideal quando temos **opções fixas** (como menus), tornando o código mais legível que múltiplos if/elif.

```
# Exemplo de Menu de Opções
opcao = int(input("Escolha: 1-Cadastrar, 2-Excluir, 3-Sair: "))
match opcao:
    case 1:
        print("Iniciando Cadastro...")
    case 2:
        print("Removendo Registro...")
    case 3:
        print("Saindo do sistema.")
    case _:
        print("Opção inválida.")
```

```
print("Opção inválida.")
```

4. Modularização com Funções e Bibliotecas

Modularizar é quebrar um problema grande em partes menores e reutilizáveis.

Anatomia de uma Função

- **def:** Palavra-chave de definição.
- **Nome:** Identificador descritivo em *snake_case*.
- **Parâmetros:** Dados de entrada que a função recebe (opcionais).
- **Corpo:** Bloco de código indentado com a lógica.
- **Return:** Comando que encerra a função e envia o resultado para quem a chamou.

Diferença Crítica: Print vs. Return

- **print()** apenas exibe um valor no terminal.
- **return** entrega o dado para o programa, permitindo que o valor seja guardado em variáveis ou usado em cálculos futuros.

Escopo de Variáveis

- **Local:** Variáveis criadas dentro de uma função existem apenas ali dentro.
- **Global:** Variáveis fora das funções. **Boa prática:** Evite modificar variáveis globais dentro de funções; prefira passar parâmetros e usar o **return**.

Bibliotecas Úteis

Módulo, Recursos Comuns

`math`, `"sqrt()"` (raiz), `ceil()` (teto), `floor()` (pisso), `math.pi` (3.1415...)"
`random`, `"randint(a, b)"` (inteiro aleatório), `choice(lista)` (item aleatório)"

5. Estruturas de Repetição (FOR e WHILE)

Escolha da Estrutura Correta

Situação, Recomendação, Ferramenta

"Iterar sobre uma coleção (lista, string)", Percorrer elementos, `for item in colecao`:

Repetir um número fixo de vezes, Repetição contada, `for i in range(qtd)`:

Repetir enquanto uma condição for verdadeira, Condição de estado, `while condicao`:

Número de repetições desconhecido, Laço condicional, `while True`:

Função `range()`

A sintaxe é `range(inicio, fim, passo)`. **Atenção:** O valor de **fim** é sempre **exclusivo** (o laço para antes de atingir esse número).

Comandos de Controle

- **break:** Interrompe e sai imediatamente do laço atual.
- **continue:** Pula o restante do código na iteração atual e vai para a próxima.

6. Manipulação de Strings (Texto)

Strings são sequências imutáveis de caracteres.

Indexação e Fatiamento

- **Indexação:** texto0 é o primeiro caractere; texto-1 é o último.
- **Fatiamento:** texto[inicio:fim:passo]. Ex: texto0:5 pega do índice 0 ao 4.

Métodos Essenciais

- `.upper()` / `.lower()`: Converte para maiúsculas/minúsculas.
- `.strip()`: Remove espaços inúteis nas bordas.
- `.replace(antigo, novo)`: Substitui trechos de texto.
- `.count(termo)`: Conta quantas vezes o termo aparece.
- `.split(sep)`: Divide a string em uma lista.

Formatação com f-strings

Permite inserir variáveis e formatar casas decimais.

```
nome = "Maria"
media = 8.5678
# O :.2f arredonda para 2 casas decimais
print(f"Aluna: {nome} | Média: {media:.2f}")
```

7. Estruturas de Dados Colecionáveis

Listas ()

Coleções ordenadas e **mutáveis**.

- **Métodos:** `append()` (adiciona ao fim), `pop()` (remove por índice), `remove()` (remove por valor), `sort()` (ordena).

Dicionários {}

Estruturas de **chave-valor**. **Regra:** As chaves devem ser de tipos **imutáveis** (strings, inteiros ou tuplas).

- **Acesso Seguro:** `dicionario.get("chave", "N/A")` evita erros se a chave não existir.
- **Métodos:** `.keys()` (chaves), `.values()` (valores), `.items()` (pares chave-valor).

Tuplas (()) e Conjuntos ({})

- **Tuplas:** Imutáveis. Usadas para registros fixos (ex: coordenadas = (-23.5, -46.6)).
- **Conjuntos (Sets):** Elementos únicos e sem ordem.
- **Operações:** `|` (União), `&` (Interseção), `-` (Diferença), `^` (Diferença Simétrica - elementos que não estão nos dois ao mesmo tempo).

Estruturas Compostas

Exemplo de Lista de Dicionários (comum para bancos de dados):

```
usuarios = [
    {"nome": "Ana", "idade": 25},
    {"nome": "Bob", "idade": 30}
]
```

```
for user in usuarios:
    print(user["nome"])
```

8. Entrada/Saída de Arquivos e Exceções

Entrada e Casting

A função `input()` sempre retorna uma `str`. Converta para `int()` ou `float()` conforme necessário.

Manipulação de Arquivos

Utilize o gerenciador `with` para garantir que o arquivo seja fechado automaticamente. O parâmetro `encoding="utf-8"` é essencial para evitar erros com acentos e caracteres especiais.

- **Modos:** `r` (leitura), `w` (escrita/sobrescrever), `a` (anexar/adicionar ao fim), `r+` (leitura e escrita).

```
with open("dados.txt", "r", encoding="utf-8") as arquivo:
    conteudo = arquivo.read()
```

Tratamento de Exceções (try/except)

Usado para capturar erros e evitar o fechamento abrupto do programa. **Dica:** Evite o uso de `except:` genérico (sem especificar o erro); isso pode ocultar bugs reais. | Exceção | Causa Comum || ----- | ----- || `ValueError` | Conversão de tipo inválida (ex: `int("abc")`). || `FileNotFoundError` | Tentativa de ler um arquivo que não existe. || `ZeroDivisionError` | Divisão de um número por zero. || `IndexError` | Acesso a um índice inexistente em uma lista. |

9. Metodologia de Revisão: Ciclo de Vida e Teste de Mesa

Ciclo de Vida das Variáveis

Compreender onde inicializar variáveis é vital para evitar erros de acumulação indevida:

1. **Persistentes:** Inicializadas **fora** do laço (pertencem ao conjunto/total).
2. **Com Reset:** Inicializadas **dentro** do laço, no início (pertencem a cada item individual).
3. **Flags:** Variáveis booleanas (`True/False`) que registram se um evento ocorreu durante a repetição.

Teste de Mesa (Trace Manual)

O teste de mesa consiste em simular o computador, anotando o estado das variáveis linha por linha. **Modelo de Tabela de Rastreio (Trace Table):** | Passo | n lido | Condição | Variável_X após | Saída || :--- | :--- | :--- | :--- | :--- || 1 | 5 | 5 != 0 (T) | 5 | - || 2 | -3 | -3 != 0 (T) | 2 | - || 3 | 0 | 0 != 0 (F) | 2 | "Final: 2" |